

PRONTUÁRIO: INDICAR NO ZIP

Dia: 28/04/2022

Horário: 08h00 – 11h30

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código previamente implementado. ✓ Não coloque nome na prova ou projeto. ✓ Atribui-se nota zero à prova em desacordo com o item acima.	✓ A prova deve ser nomeada da seguinte forma: PRONTUARIO_P1, com o “SC”. ✓ Não envie apenas as classes, mas todo o projeto. ✓ Envie o projeto como zip no Moodle.
---	---

Considere o problema especificado a seguir:

Em 2022, a Receita Federal espera receber 34,1 milhões de declarações de ajuste anual do Imposto de Renda de Pessoa Física (IRPF). A empresa na qual você trabalha está desenvolvendo um sistema para esse nicho de mercado, que simula o valor a ser pago no momento da declaração. No IRPF, a declaração pode ser feita de duas formas: Declaração Simplificada e Declaração Completa. O valor do imposto utilizando a Declaração Simplificada é calculado aplicando uma alíquota de 20% sobre o valor em ganhos tributáveis que ultrapassar R\$22.847,88 no ano anterior, menos o valor total já pago pelo contribuinte ao longo dos meses do ano anterior. Já o valor do imposto na Declaração Completa também leva em conta o valor dos ganhos tributáveis, mas possui uma alíquota variável e crescente¹. As alíquotas da Declaração Completa incidem apenas sobre sua faixa, não sobre o valor total. Por exemplo, se uma pessoa recebe R\$35.000,00 por ano, deverá pagar 15% apenas sobre o que passar de R\$33.919,92, mais 7,5% para o que ficar entre R\$22.847,88 e R\$33.919,92, sendo isento no valor abaixo disso. Na Declaração Completa é possível descontar, além do imposto já pago ao longo do ano anterior, despesas dedutíveis, tais como gastos com saúde e educação. O Limite com deduções em educação é de R\$2000,00 por ano, enquanto o valor máximo para deduções com saúde é de R\$1500,00. O arquiteto de software do seu time, bastante experiente, criou as classes do modelo usando o *design pattern* chamado *Template Method*, que usa polimorfismo para criar uma solução na qual é possível obter o valor do imposto de renda de um contribuinte seguindo sempre a mesma lógica de cálculo (*método template*), considerando, ao mesmo tempo, as especificidades de cada tipo de declaração (definidas nas subclasses). De acordo com esse *design pattern*, um método com implementação *template* **não poderá** ser alterado pelas subclasses; um ou mais métodos usados pelo *template* **deverão** ser sobrescritos para permitir a variação de comportamento; e eventuais métodos com implementação padrão ou vazia (métodos gancho) na superclasse **poderão** ser sobrescritos pelas subclasses para adicionar comportamentos opcionais ao *template*. O modelo proposto é ilustrado na Figura 1.

A interface console a ser desenvolvida deverá permitir a inserção de informações sobre renda tributável anual, valor já pago na fonte e, também, eventuais entradas para os gastos dedutíveis. Ao selecionar a opção para simular o valor do imposto de renda, o sistema deverá imprimir as informações cadastradas acima, além dos valores a pagar segundo os modelos de declaração simplificada e completa. Sua aplicação deverá simular o armazenamento de dados em banco de dados, mas utilizando estruturas de dados em memória.

¹ 7,5% <R\$22.847,88- R\$33.919,92>; 15% < R\$33.919,93 - R\$ 45.012,72>; 22,5% <R\$45.012,73 - R\$55.976,16> e 27,5% <acima de R\$55.976,16>.

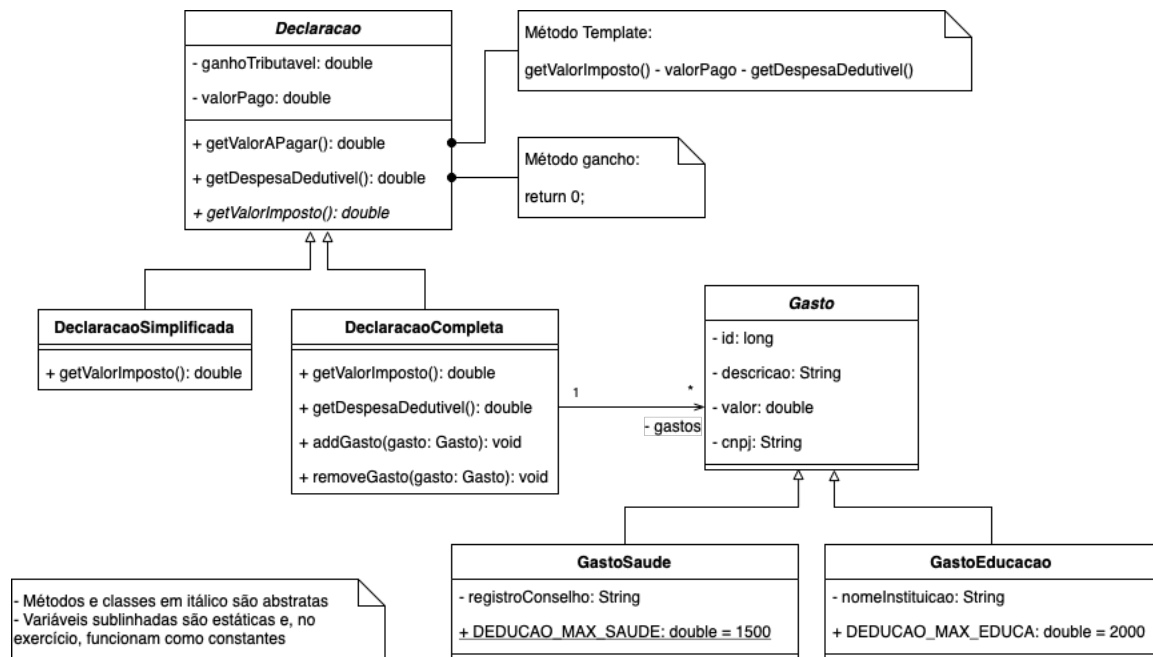


Figura 1 - Modelo da solução com Template Method

Execute as atividades a seguir para a implementação do exercício em Java. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução. Bom senso faz parte da prova.

#	Descrição	Pont.
1	Crie as classes do modelo utilizando os modificadores de acesso mais adequados aos métodos e atributos segundo o conceito de encapsulamento. Crie os construtores que achar mais adequado.	0,5pt
2	Estruture as associações de composição das classes do modelo. Para relacionamentos do tipo 1 para *, utilize listas e crie métodos para adição e remoção da lista.	0,5pt
3	Estruture os relacionamentos de herança das classes do modelo e sobrescreva o(s) método(s) pertinente(s). Forneça implementações para o(s) método(s) sobrescrito(s) de acordo com o proposto na especificação.	2,0pt
4	Implemente os métodos toString(), equals() e hashCode() adequadamente.	0,5pt
5	Aplique modificadores para obrigar, permitir ou proibir a sobrescrita de métodos da superclasse pelas subclasses, conforme indicado em negrito na especificação.	0,5pt
6	Crie classes concretas para simular a gestão da persistência das entidades. Para isso, utilize objetos do tipo Map. Quando pertinente, utilize a classe Optional para métodos nos quais o retorno de um valor é opcional. Caso tenha realizado o Item Extra 1, faça essa(s) classe(s) implementar(em) a interface genérica. Dica: o objeto Map representando a tabela do banco de dados deve ser o mesmo para toda aplicação.	1,5pt

7	Implemente funcionalidade(s) para adicionar novas instâncias da(s) entidade(s) do modelo a partir do console, sempre obedecendo restrições descritas na especificação.	1,0pt
8	Implemente funcionalidade(s) para editar novas instâncias da(s) entidade(s) do modelo a partir do console, sempre obedecendo restrições descritas na especificação.	1,0pt
9	Implemente funcionalidade(s) para remover novas instâncias da(s) entidade(s) do modelo a partir do console, sempre obedecendo restrições descritas na especificação.	0,5pt
10	Implemente funcionalidade(s) para listar uma dada instância de entidade do modelo ou todas as instâncias existentes dessa entidade. Utilize polimorfismo para representar variações de comportamento de subclasses da mesma classe.	1,0pt
11	Implemente as funcionalidades de forma a prever e mitigar possíveis exceções. Utilize adequadamente as exceções da linguagem Java. Crie ao menos uma exceção sua, para um caso pertinente.	1,0pt
Extra	Desenvolva uma classe abstrata genérica chamada GenericDAO<K, T>, capaz de emular a persistência de objetos de qualquer classe que implemente a interface Entidade<K> (também criada nesse item) em um banco de dados simulado por um Map<K,T>. Substitua as implementações feita nas classes criadas no Item 6 pela implementação padrão herdada de GenericDAO.	1,0pt +  <i>thug life</i>
	Não seguir as orientações sobre a criação e entrega do projeto descritas no preâmbulo da prova.	-1,0pt

Prêmio Usain Bolt: O aluno que terminar todas as atividades corretamente primeiro ganha 1pt adicional para usar na segunda prova. Você está voando?

*** Boa sorte! ***